

StepCoder：用编译器反馈的强化学习改进代码生成（中文精读导读）

原作者：Shihan Dou, Yan Liu, Haoxiang Jia, Limao Xiong, Enyu Zhou, Wei Shen 等
Fudan NLP Lab, Fudan University（复旦大学）等机构

英文原标题：StepCoder: Improve Code Generation with Reinforcement Learning from Compiler Feedback
发表：ACL 2024 (arXiv:2402.01391) **引用数：**较高（本次未取到精确数值）
代码与数据集：https://github.com/Ablustrund/APPS_Plus
一句话定位：让大模型写代码时，把”代码能不能跑通单元测试”当作奖励信号来做强化学习；并用两招——把”从零写整段难代码”拆成”由易到难的代码补全课程”（CCCS）、只对真正被测试执行到的代码**算梯度（FGO）**——解决”探索难”和”优化不准”两个老大难问题。

说明：本文是面向 AI 应用开发实习生的**中文精读导读**（转化性学习材料），以讲解、转述、关键句翻译和公式拆解为主，并非原文逐字全译。术语首次出现时括注英文原文，模型/数据集/方法专名保留英文。

1 小白知识卡：读这篇前要懂的几个概念

这篇论文把”代码生成”问题套进了”强化学习（reinforcement learning, RL）”的框架。如果你对 RL 还不太熟，先把下面几个概念过一遍，后面就好读了。

代码生成（code generation）

给模型一段自然语言的需求描述（比如”写一个统计子序列个数的函数”），让它自动生成能正确运行的源代码。本文用的”对错”标准很硬核：生成的代码要能通过预先准备好的**单元测试（unit test）**——也就是给定输入，输出必须和标准答案一致。

强化学习（RL）三件套：状态 / 动作 / 奖励

RL 是”在环境里不断试，根据奖励调整策略”。在写代码这个场景里：**动作（action）** = 模型每生成一个 token（词元）；**状态（state）** = 已经生成的前面那串 token；**奖励（reward）** = 整段代码写完、丢进编译器跑完单元测试后，告诉你”过了 / 没过 / 报错”。负责生成的模型叫**策略模型（policy model）**。

PPO（近端策略优化，Proximal Policy Optimization）

目前最常用的 RL 训练算法之一，也是 RLHF（基于人类反馈的强化学习）里的主力。它让策略模型”小步快走”地朝高奖励方向更新，同时用一个 KL 惩罚项拴住它、不让它跑得离原始模型太远（防止训崩）。本文就是用 PPO 来训练代码模型的。

稀疏奖励（sparse reward）与探索（exploration）难题

”稀疏奖励”是指：只有把**整段**代码生成完、跑完测试，才会拿到一次奖励信号；中间每写一个 token 都没有任何反馈。需求越复杂，要写的代码越长，模型靠”瞎试”凑出一份能通过测试的代码的概率就越低——这就是”探索难”。本文的 CCCS 就是专门治这个的。

优势函数（advantage）与信用分配（credit assignment）

RL 里用**优势函数** A 衡量”某个动作比平均水平好多少”。难点在于：一段代码里有很多 token，但单元测试可能根本没执行到其中一部分。如果对所有 token 一视同仁地算优势、算梯度，就会把”功劳/锅”算到没参与的代码头上——这叫信用分配不准。本文的 FGO 就是专门治这个的。

课程学习（curriculum learning）

像教小孩一样，**先学简单的、再学难的**。本文把”从零写一整段复杂代码”这件难事，先变成”我给你大半段标准答案、你只补最后一点点”的简单填空，再逐步减少给的提示、加大要补的比例，直到模型能完全独立写出整段——这就是 CCCS 的核心思路。

2 摘要全译

大语言模型（large language models, LLMs）的发展极大推动了代码生成领域。已有工作把强化学习（RL）与编译器反馈结合起来，借此探索 LLM 的输出空间、提升代码生成质量。然而，LLM 面对复杂人类需求时生成的代码往往很长，这使得 RL 的探索成为一大挑战；同时，由于单元测试可能覆盖不到全部复杂代码，用这些”未被执行的代码片段”来优化 LLM 是无效的。为应对这些挑战，我们提出 StepCoder——一个用于代码生成的全新 RL 框架，由两个主要组件构成：CCCS 通过把”长序列的代码生成任务”分解为”由一系列代码补全子任务组成的课程（Curriculum of Code Completion Subtasks）”来解决探索难题；FGO 则通过屏蔽（mask）未被执行的代码段，只用真正执行到的代码来优化模型，从而实现细粒度优化（Fine-Grained Optimization）。此外，我们还构建了用于 RL 训练的 APPS+ 数据集，并经过人工核验以确保单元测试的正确性。实验结果表明，我们的方法提升了探索输出空间的能力，并在相应基准上超越了当前最先进的方法。我们的数据集 APPS+ 与 StepCoder 已在线公开。

3 逐节精读

3.1 引言 (Introduction): 要解决什么问题

代码生成（也叫程序合成 program synthesis）的目标，是根据自然语言描述自动生成符合要求的源代码。基于 LLM 的方法在这件事上已经表现亮眼，但在“对齐复杂人类需求”上仍有差距。

一个很有潜力的方向是**从编译器反馈中学习**：把代码丢进编译器、跑单元测试，结果直接告诉你这段程序功能上对不对。前人（如 PPOCoder、RLTF、CodeRL）已经把“单元测试通过与否”当作奖励，用 RL 来引导模型探索输出空间。但作者指出，用 RL 训代码模型有三个拦路虎：

1. **探索难**：需求越复杂，代码越长，稀疏奖励下模型很难“探”到一份能通过测试的解。
2. **优化不准**：单测覆盖不到的代码片段和奖励无关，如果拿整段代码去算梯度，优势估计就不准。
3. **数据集质量差**：现有数据集（如 APPS）存在缺输入/输出、标准答案跑不通、输出对不上等问题，影响 RL 学习。

针对这三点，作者的贡献是：(1) 提出 StepCoder，含 CCCS 和 FGO 两个组件；(2) 构建高质量数据集 APPS+；(3) 实验证明 StepCoder 能提升探索效率与效果，超越其他方法。

关键句直译：“CCCS makes exploration easier by breaking down the complicated goals into sub-objectives curriculum. FGO provides fine-grained optimization by only utilizing the executed code in unit tests.”（CCCS 通过把复杂目标拆成“子目标课程”让探索更容易；FGO 通过只利用单元测试中被执行到的代码来提供细粒度优化。）

3.2 动机 (Motivation): 两个难题的数学直觉

作者用 APPS 数据集里一个例子（见原文图 1）把两个难题讲清楚。他们先用**抽象语法树 (abstract syntax tree, AST)** 分析代码，找出其中的**条件语句 (conditional statement, CS)**——也就是 if/for 这类会产生新分支、增加程序复杂度的语句。

难题一：探索问题。 设 $P_\theta(\text{CS}_i)$ 表示参数为 θ 的策略模型**正确写出第 i 个条件语句**的概率。那么模型按需求正确生成整段代码的概率可以写成：

$$P \propto P_o \prod_{i=1}^3 P_\theta(\text{CS}_i) \quad (1)$$

逐符号拆解：

- P ：整段代码被正确生成的总概率（正比于右边）。
- P_o ：图中标注的条件语句之外、其它代码片段被写对的概率。

- $\prod_{i=1}^3 P_{\theta}(\text{CS}_i)$: 把每个条件语句写对的概率**连乘**起来（这个例子里有 3 个）。

直觉是：每个 $P_{\theta}(\text{CS}_i)$ 都小于 1，**连乘**会让总概率迅速变小。需求越复杂、条件语句越多，连乘项越多， P 就会**指数级地塌下去**——模型几乎不可能靠探索碰巧写对整段，这就是稀疏奖励 + 长序列下探索极难的根源。作者也指出，常见的”reward shaping（奖励塑形，更频繁地人为给奖励）”在这里行不通，因为代码必须**完整生成完**才能跑、才有奖励。

难题二：优化问题。 先看标准 RL 微调的目标：把每个 token 的预测看成一个动作 a ，更新策略模型 π_{θ} 的梯度策略目标为：

$$\max_{\theta} \mathbb{E}_{(x,y) \sim D_{\pi_{\theta}}} \left[\sum_t A_t^{\pi} \log \pi(y_t \mid y_{1:t-1}, x; \theta) \right] \quad (2)$$

逐符号拆解：

- x : 人类需求（任务描述）； y : 生成的代码（一个 episode，整段轨迹）。
- $\log \pi(y_t \mid y_{1:t-1}, x; \theta)$: 在已生成前 $t-1$ 个 token 和需求 x 的条件下，生成第 t 个 token y_t 的对数概率。
- A_t^{π} : 第 t 个动作（token）的**优势**，由 GAE（广义优势估计，Generalized Advantage Estimator）从奖励 r 算出，用来降低方差。
- 整个式子的意思：**对优势为正的 token 提高其概率、对优势为负的降低其概率**，从而让模型更倾向于高回报的写法。

问题来了：奖励只跟**被执行到的代码**有关。原文举例：某个输入下只有 75% 的代码被执行，那剩下 25% 没执行的 token 跟奖励根本无关。可是式 (2) 把**所有** token 都拉进梯度计算，就会给这些无关 token 也分配优势——导致优势估计不准、优化不精确。

3.3 方法 (Method): StepCoder 怎么做

StepCoder 的整体框架见图 1。先约定记号 (Preliminaries): 训练集 $D = \{(x_i, y_i, u_i, e_i)\}_{i=0}^N$ ，其中 x 是人类需求、 y 是标准答案 (canonical solution)、 u 是单元测试样本、 e 是通过分析 AST 得到的**条件语句列表**（每个元素记录该语句的起始位置 st 和结束位置 en ，按 st 升序排列）。标准答案 y 可表示为 token 序列 $\{a_t\}_{t=0}^T$ 。

组件一：CCCS（代码补全子任务课程）。 核心思路是把”长序列探索”拆成”一串由易到难的短任务”。具体做法：

- **从靠近终点的地方起步。** 训练早期，给模型需求 x 加上标准答案的前面一大段 $x_p = \{a_i\}_{i=0}^{s^*}$ 当提示，让模型只补完剩下一小段。这里 s^* 是探索的起始点。把拼好的输入记为 $x' = (x, x_p)$ ，模型输出轨迹 τ ，最终序列 $\hat{y} = (x_p, \tau)$ 。因为只需补一点点，探索一下就成功了，奖励信号容易拿到。

- **奖励函数（沿用前人设置）：**

$$r(x', \hat{y}) = \begin{cases} +1, & \text{若 } \hat{y} \text{ 通过全部单元测试} \\ -0.3, & \text{若 } \hat{y} \text{ 有任一单元测试失败} \\ -0.6, & \text{若 } \hat{y} \text{ 发生运行时错误 (runtime error)} \\ -1, & \text{若 } \hat{y} \text{ 发生编译错误 (compile error)} \end{cases} \quad (3)$$

直觉：通过给正分；越“早死”（编译就过不了）罚得越重，引导模型至少先写出能跑的代码。

- **用 PPO 优化，并屏蔽提示部分。**在优化阶段，当作提示喂进去的标准答案片段 x_p 要被屏蔽 (mask)，不参与梯度——因为那段不是模型自己写的，不该算它功劳。目标函数为：

$$\text{Objective}(\theta) = \mathbb{E}_{(x', \hat{y}) \sim D_{\pi_\theta}} [r(x', \hat{y}) - \beta \log (\pi_\theta(\hat{y} | x') / \pi_{\text{ref}}(\hat{y} | x'))] \quad (4)$$

其中 π_{ref} 是 PPO 里的参考模型（由 SFT 模型初始化）， β 控制 KL 惩罚强度——这一项就是前面知识卡说的“拴住策略模型别跑太远”。

- **起点逐步前移（课程推进）。**随训练进行，起始点 s^* 慢慢往标准答案的开头移动。每个样本设一个阈值 ρ ：当模型在该样本上“累计正确比例”超过 ρ ，就把起点往前挪一格（给的提示更少、要补的更多）。到训练后期 $s^* = 0$ ，模型完全只靠需求 x 自己写整段代码——这时就退化成了标准 RL，但已经“学会怎么走到终点”了。
- **采样次数与条件语句挂钩。**条件语句越多 $\Rightarrow$ 独立路径越多 $\Rightarrow$ 逻辑越复杂，就需要更多次采样。作者把第 i 个样本的课程数设为 $\lceil \sqrt{E_i} \rceil$ (E_i 是其条件语句数)，课程步长设为 $\lceil [E_i] / \lceil \sqrt{E_i} \rceil \rceil$ 而非 1，以此加速训练。

CCCS 一句话总结：从靠近目标的状态出发探索很容易；从远离目标的状态出发很难，但只要能复用“已经学会怎么抵达目标”的状态，就会变容易。这正是课程学习的精髓。

组件二：FGO（细粒度优化）。作者观察到：代码生成和 Atari 这类 RL 任务不同——可以明确排除一批和奖励无关的动作。既然编译器反馈只跟“被执行到的代码片段”有关，那就在计算损失时，把没被单元测试执行到的 token 屏蔽掉，只用执行到的 token 来更新策略模型。这样优势估计就只落在“真正影响结果”的代码上，优化更精准。

完整算法（Algorithm 1）。CCCS 与 FGO 合在一起的训练循环大致是：对每个样本，先按当前课程算出起点位置 $\text{pos} = s_i \times c_i$ ，把“需求 + 标准答案前 pos 个 token”拼成新上下文 x'_i (CCCS)；采样轨迹、用式 (3) 算奖励；再算出“未执行片段的掩码矩阵” m_{ij} （执行到记 1、否则记 0，FGO）；用 PPO 更新模型；最后用滑动平均更新每个样本的通过率 ρ_i ，一旦超过阈值 ρ_t 就把课程 c_i 往前推进一阶。如此循环。

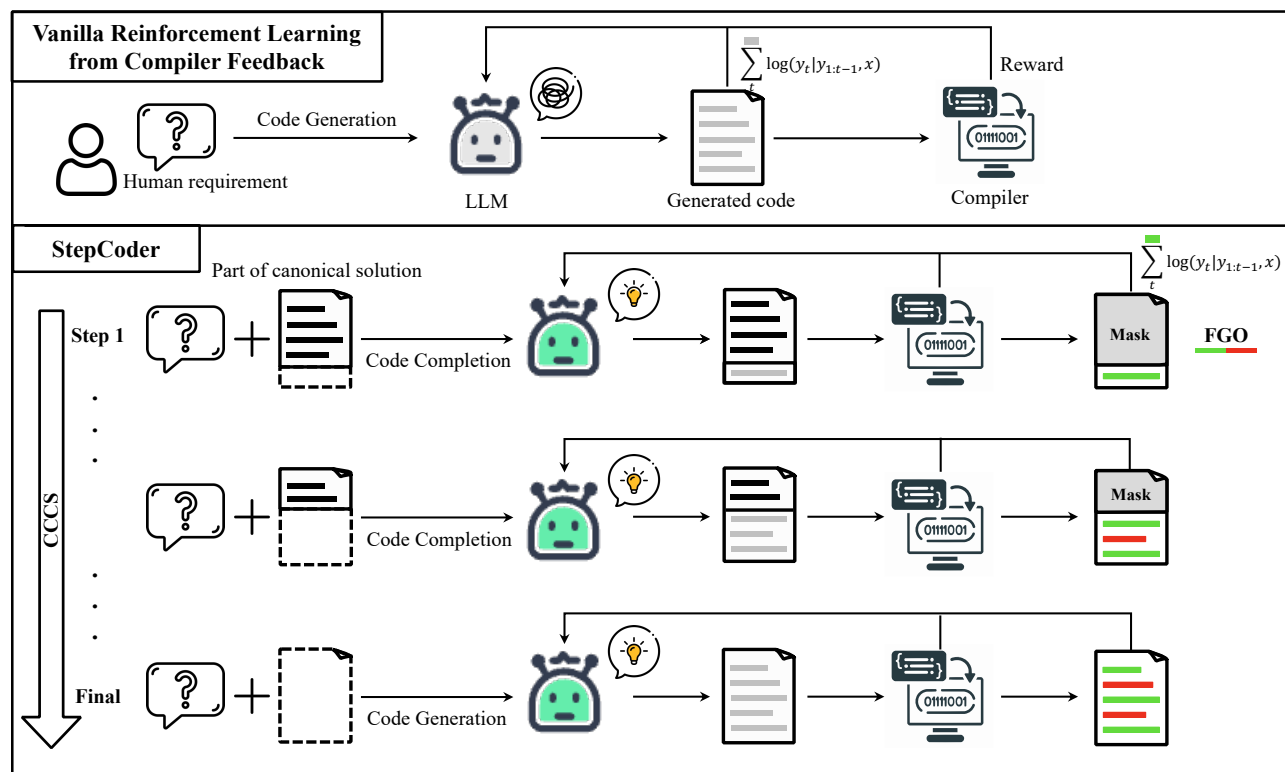


图 1: StepCoder 方法总览 (原文 Figure 2 的矢量重制)。上半部分是普通的“编译器反馈强化学习 (Vanilla RL)”；下半部分是 StepCoder。

图 1 在说什么 (数据怎么流动)。

- **上排 (Vanilla RL):** 人类需求 → LLM 生成整段代码 → 编译器跑测试给出一个 Reward。代码很长、只在最后拿一次稀疏奖励，所以探索很难。
- **下排 (StepCoder):** 左侧大箭头是 CCCS 的课程进程。Step 1 时，输入 = 需求 + 一大段标准答案 (图中实线文档表示“给定的提示”，虚线框表示“待补全的部分”)，模型只做**代码补全 (Code Completion)**，很容易成功；越往下走，给的提示越少、虚线区域越大，难度递增；到 **Final** 行，提示几乎为空，模型要直接做完整的**代码生成 (Code Generation)**。
- **右侧的 Mask 与 FGO:** 每行生成的代码送进编译器后，会得到一个掩码——图中**绿色**表示被执行到的代码、**红色**表示没被执行到的代码。**FGO 只对绿色 (执行到的) token 计算梯度** $\sum_t \log \pi(y_t | y_{1:t-1}, x)$ ，红色部分被屏蔽掉，从而实现细粒度优化。

4 实验解读

4.1 数据集：APPS+ 怎么来的

作者发现现有开源数据集里只有 APPS 勉强够格做 RL 训练,但它有不少脏数据:缺输入/输出/标准答案、标准答案编译不了或跑不起来、输出对不上等。于是他们做了一轮细致清洗:剔除缺字段的实例;统一输入输出格式以保证执行和比较时的确定性;对每个实例**跑单元测试 + 人工分析**,删掉代码不全/不相关、语法错误、API 误用、缺依赖库的;输出对不上的则人工复核题面、修正期望输出或删除。

最终得到 **APPS+**,共 **7,456** 个实例,按难度分三档:Introductory (入门,2,850)、Interview (面试,4,020)、Competition (竞赛,586)。平均每题描述 255.3 词、代码 21.9 行、附带 3 个单元测试,并标注了条件语句的起止位置。

4.2 基准、基线与设置

- **评测基准**:自建的 APPS+,外加两个公认基准 MBPP (Mostly Basic Programming Problems, 974 题入门级) 和 HumanEval (164 道手写 Python 题)。MBPP/HumanEval 用**零样本 (zero-shot)** 评测——模型只在 APPS+ 上微调,然后直接去这两个基准上考。
- **基线模型**:基座模型(CodeLlama 系列、DeepSeek-Coder-Base)、SFT 模型(StarCoder、CodeLlama-Instruct、WizardCoder、DeepSeek-Coder-Instruct 等),以及 RL 类方法 Vanilla PPO、PPOCoder、RLTF。为公平比较,所有 RL 方法都用同一个基座 **DeepSeek-Coder-Instruct-6.7B**。
- **评价指标**: Pass@k——对每个需求生成 k 份代码,只要有一份通过全部单元测试就算“过”。本文主要看 **pass@1**。
- **关键超参**: PPO 阶段策略模型学习率 $5e-7$ 、critic 模型 $1.5e-6$;每个样本采样 16 份;KL 惩罚系数 $\beta = 0.05$ 、clip 值 0.8;最大输出长度 1024。

4.3 主要结果

在 APPS+ 上 (表 1)。RL 类方法整体优于基座模型和 SFT 模型,说明用编译器反馈做 RL 确实能更好地“导航”输出空间。**StepCoder** 全面领先所有基线,Overall 达 36.1,比最强的 RL 基线 RLTF (32.7) 高出 3.4 个百分点;在竞赛级 (Competition) 这种最难的题上也拿到最高分 8.6。值得注意:在同一基座上做 SFT (“SFT on APPS+”, Overall 29.8) 几乎没比基座 (29.2) 涨多少,反衬出**编译器反馈 RL 比单纯的下一 token 预测更能提升代码质量**。

表 1: 在 APPS+ 上的 pass@1 结果 (节选自原文 Table 1)。RL 类方法统一以 DeepSeek-Coder-Instruct-6.7B 为基座。

模型	规模	Introductory	Interview	Competition	Overall
基座模型 (<i>Base Models</i>)					
CodeLlama	13B	18.7	11.0	0.0	13.0
CodeLlama-Python	13B	29.0	12.3	2.9	17.9
DeepSeek-Coder-Base	6.7B	13.0	10.3	5.0	10.9
监督微调模型 (<i>SFT Models</i>)					
StarCoder	15.6B	6.3	4.1	0.7	4.7
CodeLlama-Instruct	13B	33.3	11.0	1.4	18.7
WizardCoder-Python-V1.0	13B	39.7	15.1	4.3	23.6
DeepSeek-Coder-Instruct	6.7B	49.4	18.7	3.6	29.2
SFT on APPS+	6.7B	50.1	19.0	6.4	29.8
RL 类方法 (基座 = <i>DeepSeek-Coder-Instruct-6.7B</i>)					
Vanilla PPO	6.7B	53.7	20.1	5.0	31.7
PPOCoder	6.7B	54.4	20.3	6.4	32.1
RLTF	6.7B	55.1	20.8	6.4	32.7
StepCoder (本文)	6.7B	59.7	23.5	8.6	36.1
w/o CCCS	6.7B	58.7	21.7	7.1	34.6
w/o FGO	6.7B	58.4	23.3	8.6	35.5

消融实验 (Ablation)。 看表 1 最后两行: 去掉 CCCS (w/o CCCS) 后 Overall 从 36.1 掉到 34.6, 且竞赛级从 8.6 大跌到 7.1——印证 **CCCS 主要帮在最难的复杂题上** (因为它简化了复杂需求的探索); 去掉 FGO (w/o FGO) 后 Overall 掉到 35.5——说明 **FGO 通过把编译器反馈对准“被执行到的代码”提升了单测通过率**。两个组件都有正贡献。

在 MBPP 与 HumanEval 上 (表 2)。 模型在 APPS+ 上训练后零样本迁移到这两个基准, **StepCoder 在两个基准上都最强** (HumanEval 78.7、MBPP 67.0)。作者还担心“是不是训练集和测试集重叠才涨的”, 于是计算了 APPS+ 与两基准的代码行重叠率: 只有 0.2% (对 MBPP) 和 7.1% (对 HumanEval) 的题目有超过一半的行能在对方里匹配上——重叠极小, 说明提升是真的泛化能力, 主要来自“改善了 RL 的探索问题”。另外一个有意思的发现: **只在单一数据集上做 SFT 反而会掉点** (“SFT on APPS+” 在两个基准上都明显下降), 因为这会损害模型的指令跟随和泛化能力; 而 RL 类方法则能在未见任务上提升表现。

表 2: 在 MBPP 与 HumanEval 上的 pass@1 结果（零样本，原文 Table 2）。所有模型均在 APPS+ 上微调后评测。

模型 (6.7B)	HumanEval	MBPP
DeepSeek-Coder-Instruct	78.0	64.2
SFT on APPS+	55.5	54.8
Vanilla PPO	78.0	65.0
PPOCoder	76.8	63.8
RLTF	76.8	65.2
StepCoder (本文)	78.7	67.0

按单测结果细分（原文 Figure 4）。作者进一步分析“没通过测试”的失败案例，把失败分成“编译错误（Reward = -1）”和“运行时错误与失败（Reward = -0.6 或 -0.3）”两类。结论：StepCoder 显著降低了编译错误的比例（在面试级、竞赛级题上尤其明显），说明它更容易写出“至少能跑”的代码；不过所有 LLM 都更容易栽在运行时错误上，StepCoder 也不例外（只是相对更低）。作者由此建议：未来研究应重点降低运行时错误，这能进一步提升代码质量与通过率。

4.4 相关工作 (Related Work) 速览

- **代码生成的 LLM:** 从预训练模型 (CodeGPT、PanGu-Coder、SantaCoder、CodeGeex、Phi-1.5) 到 SFT 模型 (CodeX、StarCoder、WizardCoder、CodeLlama-Instruct、DeepSeek-Coder)；以及把 RL 引入代码生成的 CodeRL、PPOCoder、RLTF——但它们的奖励空间仍然稀疏，复杂任务探索依旧难。
- **RL 中的探索:** 用 SFT 模型初始化策略来改善探索是常见做法（本文也用了），但对复杂需求还不够；过程监督奖励模型 (process-supervised reward model) 能给逐步奖励，但需要标注大量偏好数据；也有工作用“逐步变难的起始状态”构建课程（多见于游戏、机器人）。本文的差异化在于：结合软件工程特征，用**条件语句**来动态决定起始状态，并引入 FGO 利用**代码覆盖信息**做细粒度优化。

5 核心 takeaways 与对 AI 应用开发的启示

1. “能不能跑通”是最硬的监督信号。对代码、SQL、配置、调用工具的参数这类可自动验证的任务，与其堆人工标注，不如把“执行结果对不对”做成自动奖励——这就是当下很火的 RLVR（基于可验证奖励的强化学习）思路。StepCoder 是把它在代码场景做扎实的一个范例。
2. **探索难，就上“课程”**。当目标序列很长、奖励很稀疏时，让模型“从接近答案的地方起步、逐步退到从零开始”是个通用且便宜的技巧。做 agent / 长链路任务时，可以借鉴“先给一半轨迹当提示、再慢慢撤掉”的课程式训练。

3. **信用分配要对准”真正起作用的部分”**。FGO 的本质是只对影响结果的 **token 算梯度**。在你自己的 RL/微调里，如果能识别出”哪些输出真正决定了奖励”（如被执行到的代码、被实际调用的工具、被引用的检索片段），就该把优化聚焦在那里，别让无关 token 稀释信号。
4. **数据质量决定 RL 上限**。作者花大力气清洗出 APPS+（人工核验单测、统一 I/O 格式、保证确定性比较）。一句话：**脏的单元测试 = 错的奖励 = 训歪的模型**。自动验证类项目里，先把”验证器/测试集”本身做对，比调模型更重要。
5. **单一数据集 SFT 可能伤泛化**。实验里”只在 APPS+ 上做 SFT”反而让 HumanEval/MBPP 掉点，而 RL 方法能在未见任务上提升。启示：垂直数据上猛灌 SFT 要小心**灾难性遗忘 / 指令跟随退化**，RL（带 KL 约束）相对更稳。
6. **知道短板在哪**。StepCoder 主要降的是**编译错误**，对**运行时错误**帮助有限。做评测和迭代时，按错误类型拆开看，能更精准地知道下一步该补哪块。

参考文献（节选最关键几条，原文不译）

1. Hendrycks et al. 2021. Measuring coding challenge competence with APPS. NeurIPS Datasets and Benchmarks 2021. (*APPS* 数据集，本文 *APPS+* 的来源)
2. Austin et al. 2021. Program synthesis with large language models. arXiv:2108.07732. (*MBPP* 基准)
3. Chen et al. 2021. Evaluating large language models trained on code. arXiv:2107.03374. (*HumanEval* 基准 & *Pass@k* 指标)
4. Le et al. 2022. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. NeurIPS 35. (用单测做奖励的 *actor-critic* 方法)
5. Shojaaee et al. 2023. Execution-based code generation using deep reinforcement learning (PPOCoder). arXiv:2301.13816.
6. Liu et al. 2023. RLTF: Reinforcement learning from unit test feedback. arXiv:2307.04349.
7. Schulman et al. 2017. Proximal policy optimization algorithms (PPO). arXiv:1707.06347.
8. Guo et al. 2024. DeepSeek-Coder: When the large language model meets programming. (本文 *RL* 实验的统一基座)